

Minimax Solver & Analysis

Background Information

Lots of games are been designed for fun or further exploration, while a portion of them are strategical and deterministic. The common solve for these games are interesting leading people to find the limit of designed methodology. Here we analyzed two games – Halving stone and Connect Four mini (4×4 or 5×5 grid) – and analyzed the implementation of Minimax algorithm and other ways.

Introduction

- Halving Stone
 -
- Connect Four
 - A widespread game with clear rules and simple status
 - Rule 1: Two players drop disks consequently in to the grid
 - Rule 2: All disks will drop to the bottom unless there are already disks under
 - Rule 3: Disks cannot be dropped into full columns
 - Rule 4: Any consequent 4 disks of same player that forms a diagonal, row, or column means that this player won.
 - Rule 5: When the grid is full, the 2 players tied.
 - This game is appropriate to analyze because the grid and the game status can be resembled in binary system, and a clear goal and end of game. Furthermore, each step is dependent on the opponent's approach, a sign for recursive method. This allows minimax algorithm to be implemented and interpret the end status as positive or negative goals.

Strategy & Implmentation

Halving Stone

Connect Four

Status

The current game status is represented through a tuple s ,

$$s = (\text{grid}, \text{current_player})$$

where grid is a 2D $N \times N$ list containing characters 'X' 'O' or ' ' (for empty space), and current player a string 'X' or 'O' telling whose turn it is.

Engine Operations & Logic

- `start_state()`: Initializes an empty $N \times N$ grid with 'X' moving first.
- `player(state)`: Returns the current player token stored in the state tuple.
- `actions(state)`: Identifies valid moves by checking the top row. If top-row cell c is empty, column c is a legal action.
- `next(state, action)`: Simulates gravity by placing the current player's token in the lowest available row of column action. It returns a new board state and flips the active player ('X' $\leftrightarrow$ 'O').
- `is_end(state)`: Evaluates terminal conditions: returns True if either player has connected 4 pieces in a line (horizontal, vertical, or diagonal), or if the top row is completely full (draw).
- `utility(state)`: Evaluates terminal payoffs:
 - +1 if 'X' wins.

- -1 if 'O' wins.
- 0 for a draw or non-terminal state.

Minimax Approach

This engine uses game-tree search algorithms to determine optimal moves for the AI players. The core mechanism explores the game tree recursively up to a maximum depth of 4, preventing too much scenarios to be calculated.

- Maximizing vs. Minimizing Player: Player 'X' aims to maximize the game state score (seeking $+1$), while Player 'O' aims to minimize it (seeking -1).
- Depth Limit Check: If the search reaches $\text{depth} == 0$ or a terminal state where $\text{is_end}(\text{state})$ is true, the algorithm stops expanding and returns the evaluation score via $\text{utility}(\text{state})$.
- Recursive Step:

$$\text{Minimax}(s, d) = \begin{cases} \max_{a \in A(s)} \text{Minimax}(\text{next}(s, a), d - 1) & \text{if Player}(s) = 'X' \\ \min_{a \in A(s)} \text{Minimax}(\text{next}(s, a), d - 1) & \text{if Player}(s) = 'O' \end{cases}$$

Enhancements

For performance analysis and algorithmic comparison, animations and console-clearing I/O are bypassed in favor of direct execution:

- Runs fast automated match runs
- Tracks game-level metrics (win/loss/draw ratios and average game length).
- Automatically exports structured outputs to persistent storage

Simulation Design

To determine if this game is deterministic to the methodology, we run 256 simulations, each time Minimax vs. DP in both 4×4 and 5×5 grid scenarios.

For each simulation, we collected the game length (turns) and end status, and determined the overall win rate for each player and game length.

Then, all the data were collected and stored in a json file for later use.

Simulation Results

After simulating the 2 scenarios, we found the both having a deterministic end status with optimizing method.

- For 4×4 grid, all simulations showed that the 2 algorithms tied with game length of 16 turns

Metric	Value
Grid Size	4
Total Games	256
X Wins	0
O Wins	0
Draws	256
X Win Rate	0.0

Metric	Value
O Win Rate	0.0
Draw Rate	1.0
Avg Game Length	16.0

- For 5×5 grid, all simulations showed that the second hand win with game length of 24 turns.

Metric	Value
Grid Size	5
Total Games	256
X Wins	0
O Wins	256
Draws	0
X Win Rate	0.0
O Win Rate	1.0
Draw Rate	0.0
Avg Game Length	24.0